Big Data Systems Final Project

PineGuard

TradingView Strategy Workflow Trust Layer

Monetizing TradingView strategy workflow validation

Student ID: r12323059

GitHub Repository: https://github.com/r12922076/bda_final

Live Demo: https://r12922076.github.io/bda_final/

Core thesis. PineGuard does not sell trading signals, promise returns, or execute trades. It monetizes auditable workflow validation around TradingView-style strategies by converting alert-event logs, strategy metadata, static market snapshots, evidence labels, and user-selected profile data into a paid strategy health report and monitoring dashboard.

National Taiwan University

June 2026

Contents

1 Executive Summary **2**

2 Target Customer and Wedge **2**

3 Demand Evidence and Willingness to Pay **3**

3.1 Evidence acquisition process 3

3.2 Evidence governance and limitations 3

3.3 Evidence categories 4

3.4 Willingness to pay 5

4 Data Product and Business Model **5**

4.1 Health-score rationale 6

5 Technical Architecture **6**

5.1 Deployed static SPA 6

5.2 Production architecture 7

6 Implementation and Reproducibility **8**

7 Go-to-Market Difficulties and Risk Controls **8**

8 Conclusion **9**

A References **9**

1 Executive Summary

PineGuard is a static, deployed data-product prototype for paid or power TradingView users who already use alerts, Pine Script indicators, Strategy Tester, webhooks, or paid scripts, but lack a structured validation workflow. The beachhead customer is the alert-heavy paid TradingView user: this user already generates alert-event data, already invests in trading workflow tools, and already faces the operational problem of knowing whether a strategy is credible after it leaves the chart.

Product boundary

PineGuard is workflow validation, not investment advice. Its output is a strategy health report: score breakdown, missing workflow components, forward-test outcome summary, evidence basis, and recommended next validation step. The score measures workflow completeness and evidence quality; it does not estimate expected profitability.

The deployed artifact is a static single-page application on GitHub Pages. It contains routes for a landing page, health-check wizard, diagnosis page, strategy-template library, forward-test monitor, evidence explorer, downloadable report, pricing page, architecture page, risk page, and documentation hub. The production architecture is specified separately as an event-driven system with webhook ingestion, durable queues, validation workers, relational metadata storage, raw payload storage, scheduled recomputation, and report delivery.

2 Target Customer and Wedge

The initial customer is an individual power user or small trading team that uses TradingView as the front end for discretionary or semi-systematic decision support. The first wedge is narrower than the full TradingView user base:

Table 1: Beachhead and expansion segments

Segment	Current workaround	PineGuard product
Alert-heavy paid TradingView user	Configures many alerts, then manually checks charts, screenshots, spreadsheets, or memory to judge outcomes.	Strategy Health Check and Forward-Test Monitor.
Pine Script builder	Uses Strategy Tester and code inspection but may miss repainting, lookahead, or unrealistic execution assumptions.	Health Check and Pro Completion Plan.
Paid-script buyer	Evaluates public claims or invite-only scripts with limited source-code visibility.	Later expansion: due-diligence checklist.
Indicator creator	Wants credibility artifacts without promising returns.	Later expansion: Creator Trust Report.

The product starts with alert-heavy users because they already produce the event data PineGuard needs. This makes the first monetizable workflow concrete: import or forward alerts, evaluate future outcomes, diagnose missing workflow components, and generate a report. PineGuard therefore acts as a trust layer between TradingView strategy development and any later execution bridge or manual trading decision.

3 Demand Evidence and Willingness to Pay

The demand case uses public digital traces rather than private interviews. The evidence dataset contains official documentation, public script or marketplace traces, competitor or adjacent product pages, pricing references, and manually coded public evidence. The evidence is directional, not a representative survey. It is still useful because it distinguishes direct technical evidence from indirect willingness-to-pay benchmarks. The report treats public-evidence-derived personas as synthetic workflow summaries rather than interview respondents.

3.1 Evidence acquisition process

Table 2: Evidence acquisition and coding workflow

Step	Procedure	Output
Seed selection	Start from TradingView official pages, alert/webhook documentation, Pine Script documentation, public scripts, competitor pages, and pricing pages.	Seed URL table.
Collection	Keep public pages only and avoid login-only, private, or paywalled content.	Raw public evidence rows.
Manual coding	Assign source type, pain category, relevance note, evidence role, and strength label.	Combined evidence snapshot.
Build step	Convert CSV evidence, alert events, strategy templates, and market snapshots into a single static JSON file.	Runtime data for GitHub Pages SPA.
Validation	Run static-site and frontend smoke tests.	Reproducibility checks.

Evidence strength is intentionally conservative. Official documentation is marked as strong technical evidence. Competitor pricing and adjacent-product pages are medium willingness-to-pay benchmarks. Marketplace listings are weaker demand proxies.

3.2 Evidence governance and limitations

The evidence supports a defensible beachhead hypothesis, not proven product-market fit. PineGuard's demand claim is deliberately conservative: paid TradingView users already generate strategy-workflow data, public sources document validation pain, and adjacent tools show that this audience pays for workflow infrastructure. A real launch would still require direct interviews, landing-page conversion tests, and retention metrics.

Table 3: Evidence types and conservative interpretation

Evidence type	What it supports	Limitation
Official platform documents	Alerts, webhooks, repainting, Strategy Tester assumptions, and Pine Script behavior are real platform-level topics.	Technical validity, not willingness to pay.
Academic or professional validation sources	Backtest overfitting, lookahead, transaction costs, and slippage are recognized strategy-validation risks.	Not always specific to retail TradingView users.
Public user discussions	Qualitative pain around backtest/live gaps, alert tracking, paid-script opacity, and manual workarounds.	Self-selected and non-representative.
Competitor pricing	Adjacent trading workflow, journal, automation, and research tools are monetized.	Indirect WTP benchmark, not PineGuard's demand curve.
Marketplace listings	Non-repaint, webhook, and backtest language appears in public strategy claims.	Promotional and weaker as demand evidence.

3.3 Evidence categories

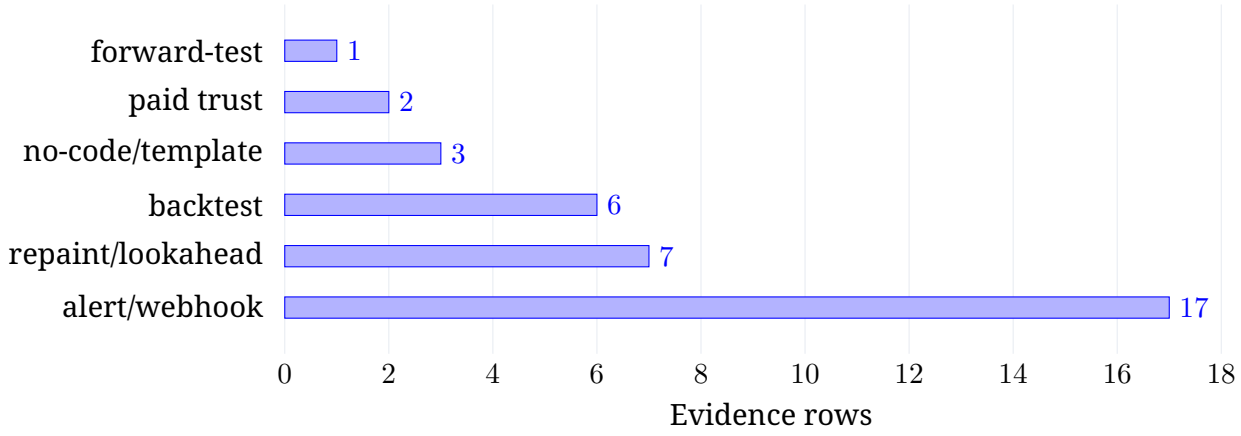

Figure 1: Evidence rows by pain category in the static evidence snapshot.

Table 4: Evidence role by strength label in the current snapshot

Evidence role	Strong	Medium	Weak	Interpretation
Technical validity evidence	14	0	0	Platform mechanics and validation failure modes are directly documented.
Adjacent-market benchmark	0	13	0	Pricing and product benchmarks support adjacent willingness-to-pay, not direct demand.
Demand proxy	0	3	5	Public pain signals are directional and non-representative.
WTP benchmark evidence	0	1	0	One row directly supports pricing logic, but it is still not a conversion test.

The strongest technical evidence concerns repainting, backtest realism, and webhook/alert workflows. TradingView documentation discusses repainting as historical-versus-realtime behavior differences[1], strategy testing assumptions such as order behavior, commission, and slippage[2], and webhook alert delivery through HTTP POST requests[3]. These sources do not prove willingness to pay, but they establish that PineGuard addresses real workflow properties rather than invented concerns.

3.4 Willingness to pay

PineGuard's price corridor is benchmarked from adjacent products rather than a direct customer survey. This is an adjacent willingness-to-pay benchmark, not direct proof of PineGuard demand. Users already pay for TradingView, webhook automation, trading journals, and strategy-testing platforms. PineGuard should price below execution-routing tools because it validates rather than routes orders.

Table 5: Adjacent-product benchmarks and PineGuard interpretation

Category	Examples	Public benchmark source	Interpretation
Trading workflow platform	TradingView paid plans[4]	Paid plans tied to alerts, indicators, history, and workflow capacity.	The target segment already pays for trading tools.
Webhook automation	TradersPost; PickMyTrade; CrossTrade[11,12,13]	Public pricing and comparison pages show monthly alert-routing infrastructure products.	PineGuard should price below execution-routing tools because it validates rather than routes orders.
Trading journals and analytics	TradeZella; Edgewonk; TraderSync[14,15]	Trading journals monetize structured post-trade analytics and reports.	Supports one-time reports and monthly monitoring around strategy workflows.
Backtesting and research	TrendSpider; QuantConnect[16,17]	Research and backtesting platforms use premium subscriptions or paid platform access.	Supports paid strategy-evaluation products, but remains an adjacent benchmark.

The proposed entry pricing is USD 19-49 for a one-time Strategy Health Check and USD 19-39/month for a Forward-Test Monitor. In non-monetary terms, PineGuard replaces manual alert review, screenshots, spreadsheet tracking, and informal paid-script due diligence.

4 Data Product and Business Model

PineGuard monetizes the structured interpretation of strategy workflow data. Raw alerts and public URLs are not the product. The paid product is the validated report and monitoring workflow. In ecosystem terms, TradingView creates and displays strategies, execution bridges can route alerts to brokers, and PineGuard occupies the missing validation layer between those two steps.

Table 6: Data-to-product mapping

Input data	Processing	Output
Alert-event logs	Join alert timestamp, direction, symbol, and future price snapshot.	Forward-test outcome table.
Strategy metadata	Check missing cost assumptions, exits, versioning, repaint review, and payload schema.	Health score and gap diagnosis.
Public evidence rows	Label source type, pain category, evidence role, and evidence strength.	Evidence Explorer and report basis.
User profile	Match style, market, risk tolerance, and concerns to validation templates.	Recommended next validation modules.

4.1 Health-score rationale

The score is intentionally a workflow-completeness score. It makes no profitability claim. The weights are fixed and auditable so users can see why a report changes when a missing workflow component is added.

Table 7: Health-score weights used in the browser-side scoring logic

Category	Max	Reason for weight
Backtest realism	25	Strategy Tester evidence is fragile when cost, slippage, and execution assumptions are missing.
Live alert observability	20	TradingView alerts become valuable data only when alert history and payload schema are captured.
Repainting/lookahead review	20	Multi-timeframe and realtime/historical differences are TradingView-specific failure modes.
Risk-rule completeness	20	A workflow is incomplete without exit logic and position-sizing assumptions.
Evidence quality	15	Forward-test records and version history make the report auditable.

5 Technical Architecture

5.1 Deployed static SPA

The deployed system is a static GitHub Pages application. It uses pre-built CSV/JSON snapshots and browser-side JavaScript. This is appropriate for the course artifact because the goal is to demonstrate the data product without requiring private backend credentials. The current prototype does not receive live webhooks, store private strategy data, or use DuckDB-Wasm at runtime; local OLAP processing is a production extension, not a claim about the demo.

Table 8: Deployed SPA routes

Route	Purpose
`#/tour`	Two-minute grader path through the deployed artifact.
`#/health-check`	Five-step wizard for persona, strategy profile, available data, and risk concerns.
`#/diagnosis`	Health score, category breakdown, and missing workflow components.
`#/monitor`	Forward-test monitor with horizon controls, sample CSV download, and webhook payload copy.
`#/evidence`	Filterable evidence explorer with source type, pain category, and evidence strength.
`#/report`	Downloadable Markdown/JSON Strategy Health Report.
`#/architecture`	Static and production architecture.

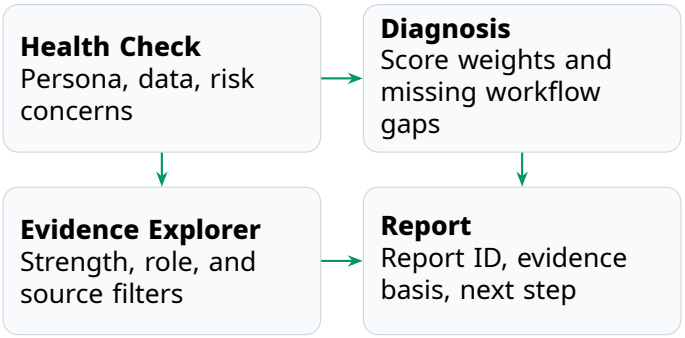

Figure 2: Deployed-route montage. The live site exposes a grader path from health-check input to diagnosis, evidence review, and downloadable report.

5.2 Production architecture

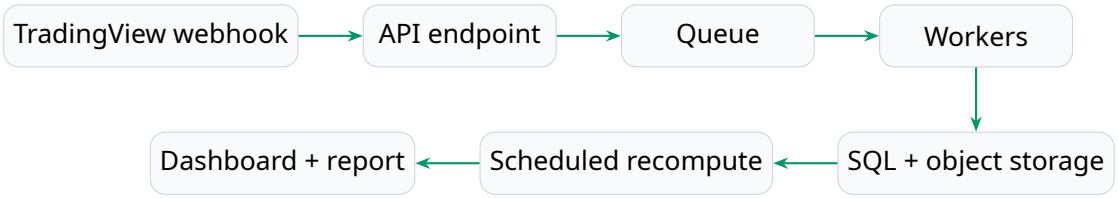

Figure 3: Production event-driven architecture. The static course artifact implements the delivery and analytics layer; production would add live authorized ingestion and persistent storage.

The production version would acknowledge webhooks quickly, enqueue raw payloads, validate idempotency, store raw events and report metadata, recompute reports on a schedule, and expose a dashboard/API/report layer. If privacy-preserving local analytics becomes important, the design can add DuckDB-Wasm or a similar browser-side analytical layer so that heavy report queries run near the user rather than in a central warehouse. At 10x scale, the main additions are a queue, relational metadata store, and daily workers. At 100x scale, the system needs partitioned event tables, object storage retention policy, retry monitoring, and cost-aware pricing for high-frequency users.

Table 9: Production webhook event contract

Field	Purpose
`event_id` and `payload_hash`	Idempotency and duplicate-event control.
`user_id`, `strategy_id`	User-authorized partitioning and report grouping.
`symbol`, `timeframe`, `signal`	Normalized strategy-event metadata.
`alert_time`, `alert_price`	Forward-test attribution anchor.
`payload_version`, `raw_payload_uri`	Schema evolution and raw audit trail.

6 Implementation and Reproducibility

The repository contains all source code, static data, configuration, and documentation needed to run the site locally or through GitHub Pages. The most important implementation files are:

Table 10: Repository structure

Path	Role
`docs/index.html`	Static SPA entry point and navigation.
`docs/assets/app.js`	Page rendering and route-specific UI logic.
`docs/assets/scoring.js`	State handling, template recommendation, forward-test attribution, health scoring, and report generation.
`docs/assets/components.js`	Reusable UI helpers, CSV parsing, downloads, and copy actions.
`docs/data/*.csv`, `*.json`	Static runtime snapshots and public evidence data.
`scripts/validate_static_site.py`	Required file and data-key validation.
`scripts/frontend_smoke_test.js`	Dependency-free route and data-contract smoke test.
`docs/grader_guide.md`	Quick path for evaluating the demo.
`docs/data_lineage.md`	Source-to-report lineage.
`docs/data_contract.md`	Required fields and runtime assumptions.
`docs/evidence_governance.md`	Evidence strength labels, interpretation limits, and conservative demand claims.
`docs/public_evidence_personas.md`	Synthetic personas derived from public evidence, explicitly not interviews.
`docs/platform...risk_notes.md`	Platform dependency, privacy, paid-script, and investment-advice boundaries.

The artifact can be run locally with `python -m http.server 8000 -d docs`. The repository-level validation commands are:

```
python scripts/validate_static_site.py
node scripts/frontend_smoke_test.js
```

7 Go-to-Market Difficulties and Risk Controls

The strongest go-to-market risk is trust. Trading-related tools can easily be misunderstood as signal sellers or performance promises. PineGuard mitigates this by positioning itself as workflow validation, not trade recommendation or execution. It also avoids publicly attacking specific paid scripts; reports are user-initiated and based on the user's own observed workflow data. Other major risks are platform dependency, data rights, privacy, cold start, and competition.

Table 11: Risks and controls

Risk	Control
Platform dependency	Use user-authorized alert payloads and user-provided exports; avoid unauthorized scraping.
Investment-advice boundary	Do not recommend securities, execute trades, or promise returns.
Privacy	Do not collect broker passwords or unnecessary personal data.
Cold start	Start with one-time health checks before monthly monitoring.
Paid-script disputes	Do not publish accusatory third-party rankings; generate private user-initiated reports from observed data.
Competition	Differentiate through evidence-backed reports, alert/outcome history, and audit trails.
Unit economics	Price high-frequency users separately because they create disproportionate storage and processing load.

8 Conclusion

PineGuard satisfies the assignment goal by connecting technical data processing to a revenue model. The monetized asset is not raw data or alpha. It is auditable workflow validation: a structured product that turns alerts, strategy metadata, market snapshots, and public evidence into a reportable workflow. The current GitHub Pages deployment demonstrates the customer-facing product, while the production architecture explains how the same product would scale with live ingestion, queues, storage, and scheduled processing.

A References

[1] TradingView. *Pine Script Documentation: Repainting*. https://www.tradingview.com/pine-script-docs/concepts/repainting/. Accessed: 19 June 2026.

[2] TradingView. *Pine Script Documentation: Strategies*. https://www.tradingview.com/pine-script-docs/concepts/strategies/. Accessed: 19 June 2026.

[3] TradingView. *How to Configure Webhook Alerts*. https://www.tradingview.com/support/solutions/43000529348-how-to-configure-webhook-alerts/. Accessed: 19 June 2026.

[4] TradingView. *Pricing and Plan Features*. https://www.tradingview.com/pricing/. Accessed: 19 June 2026.

[5] Bailey, D. H., Borwein, J., Lopez de Prado, M., and Zhu, Q. J. *The Probability of Backtest Overfitting*. Journal of Computational Finance, 2017.

[6] GitHub Docs. *GitHub Pages*. https://docs.github.com/en/pages. Accessed: 19 June 2026.

[7] Apache Kafka Documentation. *Event Streaming Concepts*. https://kafka.apache.org/documentation/. Accessed: 19 June 2026.

[8] Stripe. *Idempotent Requests*. https://docs.stripe.com/api/idempotent_requests. Accessed: 19 June 2026.

[9] DuckDB. *DuckDB-Wasm: Efficient Analytical SQL in the Browser*. https://duckdb.org/2021/10/29/duckdb-wasm. Accessed: 19 June 2026.

[10] TradingView. *Terms of Use, Policies and Disclaimers*. https://www.tradingview.com/policies/. Accessed: 19 June 2026.

[11] TradersPost. *Automated Trading Bot Platform Pricing*. https://traderspost.io/pricing. Accessed: 19 June 2026.

[12] PickMyTrade. *Automated Trading FAQ*. https://pickmytrade.io/general-faq. Accessed: 19 June 2026.

[13] Hootsuite Engineering. *A Scalable, Reliable Webhook Dispatcher Powered by Kafka*. https://medium.com/hootsuite-engineering/a-scalable-reliable-webhook-dispatcher-powered-by-kafka-2dc3d677f16b. Accessed: 19 June 2026.

[14] TradeZella. *Pricing*. https://www.tradezella.com/pricing. Accessed: 19 June 2026.

[15] TradeZella. *TradeZella vs Edgewonk and TraderSync Comparisons*. https://www.tradezella.com/vs/edgewonk. Accessed: 19 June 2026.

[16] TrendSpider. *Trading Platform and Strategy Tools*. https://trendspider.com/. Accessed: 19 June 2026.

[17] QuantConnect. *Pricing*. https://www.quantconnect.com/pricing. Accessed: 19 June 2026.

[18] Stack Overflow. *Pine Script V5: How to Consider Spread and Commission in Backtesting*. https://stackoverflow.com/questions/75297856/pine-script-v5-how-to-consider-the-spread-and-commission-in-backtesting. Accessed: 19 June 2026.

[19] TradingView. *Script Publishing Rules*. https://www.tradingview.com/support/solutions/43000590599-script-publishing-rules/. Accessed: 19 June 2026.

[20] TradingView. *House Rules and Publishing Guide*. https://www.tradingview.com/house-rules/. Accessed: 19 June 2026.